

02. TypeScript

Detailed beginner handbook - English and Arabic

Technologies: TypeScript

What you will learn

Describe a dashboard record so mistakes are caught before the browser runs.

TypeScript adds types to JavaScript. Types make autocomplete, refactoring, and API data safer.

الهدف: بناء مثال صغير وفهمه خطوة بخطوة.

شرح بسيط: تعلم الفكرة أولاً، ثم جرب المثال الصغير ببيانات اصطناعية فقط.

How to study

Study one lesson at a time. Type examples yourself, predict the result, run the code, then change one thing. Keep a notebook of new words, errors, root causes, and fixes.

Course map

Setup and mental model; ten core concepts; a guided mini-project; the real dashboard architecture; debugging; exercises and answers.

Lesson 1 - Setup and mental model

Prerequisites

Windows 10 or 11, VS Code, PowerShell, and permission to install learning dependencies. Use only synthetic data in tutorials and screenshots.

Setup sequence

1. Open PowerShell.
2. Go to learning-examples/02-typescript.
3. Read README.md.
4. Run npm install then npm run check.
5. Read the complete error if it fails.
6. Change one safe value and rerun.

الخطوات: افتح مجلد المثال، شغل الأمر، غير قيمة واحدة، ولاحظ النتيجة.

Mental model

Treat TypeScript as one layer in a system. Identify the input, the transformation or rule, the output, and possible failures. Understanding that flow is more valuable than memorising syntax.

Checkpoint

Explain: What problem does this technology solve? What is its input? What output should it produce? What can fail?

Lesson 2 - Why static type checking helps JavaScript projects

What is it?

JavaScript errors often appear only at runtime. TypeScript analyses source before execution and reports incompatible values, missing properties, unsafe calls, and incorrect assumptions.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In TypeScript, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
let total: number = 12;  
// total = 'twelve'; // Type error before runtime
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Why static type checking helps JavaScript projects.

Why static type checking helps JavaScript projects: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 3 - Primitive types, inference, annotations, and literal types

What is it?

string, number, boolean, null, and undefined are primitive types. Inference learns a type from an initial value; an annotation explicitly states the intended contract.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In TypeScript, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
const project = 'Learning'; // inferred string
let ready: boolean = true;
const fixed: 2026 = 2026;
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Primitive types, inference, annotations, and literal types.

Primitive types, inference, annotations, and literal types: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 4 - Arrays, tuples, object types, interfaces, and type aliases

What is it?

Type arrays as `Item[]`. Tuples fix positions. An interface names an object shape and can extend another; a type alias can also represent unions and primitives.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In TypeScript, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
interface Summary { month: string; total: number }  
const rows: Summary[] = [{month: 'Jan', total: 12}];
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Arrays, tuples, object types, interfaces, and type aliases.

Arrays, tuples, object types, interfaces, and type aliases: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 5 - Optional properties, readonly data, unions, and intersections

What is it?

A question mark makes a property optional; readonly prevents assignment through that type. A union allows alternatives; an intersection combines requirements from multiple types.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In TypeScript, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
type Result = {ok:true; total:number} | {ok:false; error:string};  
const value: Result = {ok:true, total:12};
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Optional properties, readonly data, unions, and intersections.

Optional properties, readonly data, unions, and intersections: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 6 - Typed functions, callbacks, promises, and async results

What is it?

Type parameter values and the return value. Promise describes an asynchronous result. Callback types specify which arguments a function receives and what it returns.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In TypeScript, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
async function load(): Promise<number> {  
    return 12;  
}  
const show = (value:number): string => `${value}`;
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Typed functions, callbacks, promises, and async results.

هذا الدرس يشرح: Typed functions, callbacks, promises, and async results. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Narrowing with typeof, in, equality, and discriminated unions

What is it?

A union must be narrowed before using member-specific operations. typeof, property checks, equality, custom predicates, and discriminant fields prove which alternative is present.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In TypeScript, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
function message(value: string | number) {  
  return typeof value === 'number' ? value.toFixed(0) : value.trim();  
}
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Narrowing with typeof, in, equality, and discriminated unions.

Narrowing with typeof, in, equality, and discriminated unions: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 8 - Generics for reusable containers and functions

What is it?

A generic type parameter preserves relationships without choosing a concrete type. function first(items:T[]):T|undefined returns the same element type supplied by the caller.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In TypeScript, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
function first<T>(items: T[]): T | undefined {  
    return items[0];  
}  
const month = first(['Jan', 'Feb']);
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Generics for reusable containers and functions.

Generics for reusable containers and functions. اكتب المثال بنفسك، حدد المدخلات والعمليات والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 9 - Modules, imports, exports, and declaration files

What is it?

export exposes a value or type; import consumes it. Declaration files describe JavaScript libraries. Use import type when an import exists only for checking.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In TypeScript, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
// summary.ts
export type Summary = {total:number};
export const empty: Summary = {total:0};
// another file: import {empty} from './summary';
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Modules, imports, exports, and declaration files.

Modules, imports, exports, and declaration files: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 10 - tsconfig, strict mode, compilation, and source maps

What is it?

tsconfig controls target JavaScript, module handling, included files, and strict checks. Strict mode catches null, implicit-any, function variance, and property initialization problems.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In TypeScript, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
// tsconfig.json
{
  "compilerOptions": { "strict": true, "target": "ES2022" }
}
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates tsconfig, strict mode, compilation, and source maps.

اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لفهم السلوك. هذا الدرس يشرح: tsconfig, strict mode, compilation, and source maps

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 11 - Typing API responses without pretending untrusted data is safe

What is it?

A type assertion does not validate network JSON. Parse unknown input with explicit checks or a schema validator before treating it as a trusted application type.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In TypeScript, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
function isSummary(x: unknown): x is {total:number} {  
  return typeof x === 'object' && x !== null && 'total' in x;  
}
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Typing API responses without pretending untrusted data is safe.

هذا الدرس يشرح: untrusted data is safe. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Guided mini-project

Project goal

Describe a dashboard record so mistakes are caught before the browser runs.

Starting example

```
type Summary = { month: string; total: number };  
const row: Summary = { month: 'January', total: 12 };  
console.log(row.total);
```

Read the code

Find imported tools or page elements. Identify the synthetic input. Find the function, event, route, or transformation that changes it. Finally, locate where the result is displayed, returned, saved, or packaged.

Build sequence

1. Run the untouched example.
2. Record the result.
3. Rename one unclear value.
4. Add a synthetic record.
5. Validate empty input.
6. Validate incorrect input.
7. Add a helpful error.
8. Rerun every case.
9. Compare with exercise-solution.
10. Explain every line.

Definition of done

Normal, empty, and invalid cases behave deliberately; no private data is used; and you can explain every line without reading the guide.

Lesson 8 - The real dashboard

Architecture connection

The application combines TypeScript with the rest of the stack. The frontend gathers filters and files; the local API validates requests; the data layer transforms workbook rows; charts present aggregates; export tools create files; and the desktop layer packages the application for Windows.

Trace the upload feature

The user selects a workbook. The frontend sends it to the local API. The backend validates the file and sheets. The data layer creates safe tables. The API returns metadata. React updates state. Plotly displays results. Identify exactly where this guide's technology participates.

Privacy and quality

Do not place narrative, contact, or identifying fields in tutorials, logs, screenshots, tests, or exports. Use generated identifiers and synthetic totals. Validate at each boundary and collect only fields required for the calculation.

Architecture exercise

Draw five boxes: UI, API, Data Processing, Export, Desktop Packaging. Add arrows and label the data. Circle the box related to this guide and add two validation checks.

Lesson 9 - Debugging

Reliable debugging loop

1. Reproduce with the smallest input.
2. Read the first meaningful error.
3. Find the exact file and line.
4. Inspect value and type.
5. Compare expected and actual results.
6. Change one thing.
7. Rerun the same case.
8. Add a regression test.

Common beginner mistakes

- Wrong folder or command.
- Missing dependency or inactive environment.
- Misspelled file, variable, field, route, or import.
- Assuming input is always present.
- Mixing setup, processing, and display in one block.
- Hiding an exception rather than understanding it.
- Using real sensitive data in a test.

أخطاء شائعة: تشغيل الأمر من مجلد خاطئ، نسيان تثبيت التبعيات، أو تغيير أشياء كثيرة بمرة واحدة.

Error notebook

Record: command, expected result, actual error, root cause, smallest fix, and test added. This turns errors into reusable knowledge.

Lesson 10 - Exercises and answers

Exercises

1. Explain the technology in three sentences.
2. Recreate the example without copying.
3. Add one normal synthetic record.
4. Handle empty input.
5. Handle invalid input.
6. Improve unclear names.
7. Add or describe one test.
8. Locate the technology in the dashboard.
9. Record one error and root cause.
10. Add one mini-project feature.

تمرين: أضف شهرا أو قيمة أمانة جديدة، ثم اشرح ما حدث بكلماتك.

Answer guide

A strong solution is observable and explainable: the documented command works; output changes for the new record; empty and invalid cases have deliberate results; names communicate purpose; a test states an expected result; and the architecture explanation identifies the correct boundary.

Next step

Next: 03. React